- Quality of Presentation (20 points)
    - All Students present with confidence, speak clearly, are strongly engaged with audience.
    - All Students can explain complex concepts articulately and concisely.
    - All Students present Information meaningfully and in logical order.
    - Strong Conclusion + Discussion of Future Work/agreed upon stubs, eg, what would the app do if there were more time in terms of additional features, security, efficiency, stubbed modules, etc.
    - The oral presentation (10 minutes) should be in three parts:
        - Overview/Introduction
            - Motivation/Purp
        - App Walkthrough
            - Role Play is a good approach!
            - Using multiple devices and showing their real-time interaction is also gr
        - Implementation Details, Including:
            - Roadblocks/Surprises/Workarounds.
            - What would you do differently?
            - Future Work.
            - List references.
- Quality of Additional Artifacts (10 points)
    - Documentation of things not in the slides/presentation, must include links and access to Git and any project management tools (Jira, Trello, etc) that was used.
    - Organized and Well Written.
    - Clearly expresses the app's functionality and the apps underlying technologies.
    - Highlights technical detail with key pieces of code described, (eg, how to interact with external APIs, Web Services, etc.).
    - Good use of graphics/screenshots of app.
    - Describes Technical Challenges.
- Small Bonuses - Optional (10 points)
    - Proper request of device permissions.
    - Proper interfaces for communication if using Fragments.
    - Use of RecyclerView or CardView where appropriate.
    - Good use of Menus.
    - Using Gestures and/or Accelerometer correctly (if needed).
    - Targeting an additional Device with an additional orientation (must do it well)
        - eg, 1 phone, 1 tablet.
    - Targeting multiple locales (must also do it well)
        - eg, English, Spanish.
    - Deploying your app to the Google Play Store.

Final Project Report

1. Team Info & Individual Contributions

Yida Wang (Sprint Master):

Calendar page design.
Viewmodels architecture and DB-related work.
Blog List implementation.
Blog deletion feature.
Report & slides.
Home Page

Jingkun Lin:

Blog Detail page implementation
Map Page implementation
Shake feature implementation.
Report & slides

Haoxuan Sun:

Home Page & Memories page: UI and logic implementation.
Navigation bar.
Language/Notification/Theme settings page design and implementation.
Polish UI design.
Report & slides & storyboard.

Every team member commits to GIT and everyone is familiar with all parts of the code although each focuses on specific areas. Yida and Jingkun fork the project, commit to their repository, and create pull requests to the main repository. All three members also commit to their own branches and merge into the main.

2. Brief vision update

Updates since the last report:

1. The blog list and picture carousels are clickable and navigate to the blog details page.
2. Search Filters on the home page are workable for time window search.
3. For the date that blogs are added in, a small dot is shown below the date on the memories page, similar to the Google calendar.
4. Users can add pictures and emojis to each blog details page. And a precious location can be bonded with each blog.
5. A new map page shows all the locations added to blogs.
6. Blogs can be deleted from the database.

7. Language is set as the same as the system's language, but users can also choose the app's language from 6 given choices.
8. Users can choose the app's theme from 4 given choices.
9. Notifications for making daily blogs can be turned on.

3. Details on Architecture & Design

Database

a.

i. What DB

SQL

- @PrimaryKey val id: UUID = UUID.randomUUID(),
- var title: String,
- val date: LocalDate,
- var text: String = "",
- var photoFileName: String? = null,
- val location: String = "",
- var emoji: String = ""

Type converters, we only have one none primitive type so only need one set to type converters:

```
fun fromDate(date: LocalDate): Long {
        return date.toEpochDay()
}
fun toDate(epochDay: Long): LocalDate {
         return LocalDate.ofEpochDay(epochDay)
}
```

ii. Where (on device, cloud)

Data is stored on the device.

Maybe the cloud storage feature as a bonus in the future.

b. ViewModels

We are using 14 different view models to manage the interaction between our views. The reason we have many viewModel is that we are handling multiple list at the same time as well as some other persistence state

c. APIs used, any changes, what's working?

Kizitonwose Calendar:

```
@Composable
fun Day(day: CalendarDay, blogDateVIewModel: BlogDateVIewModel, isSelected:
Boolean, onClick: (CalendarDay) -> Unit) {
   val colors = ThemeManager.getAppThemeColors()
```

```
    val addressMap  = blogDateVIewModel.addresses.collectAsState(initial =
mutableMapOf())
    Box(
        modifier = Modifier
            .clip(CircleShape)
            .aspectRatio(1f)
            .background(color = if (isSelected) colors.primaryVariant else Color.Transparent)
            .clickable(
                enabled = day.position == DayPosition.MonthDate,
                onClick = { onClick(day) }
            ),
        contentAlignment = Alignment.Center
    ) {
        Column(){
            Text(
                text = day.date.dayOfMonth.toString(),
                color = if (day.position == DayPosition.MonthDate) Color.Black else Color.Gray
            )
            Box(
                modifier = Modifier
                    .size(5.dp)
                    .clip(CircleShape)
                    .align(Alignment.CenterHorizontally) // Align horizontally
                    .background(color = if (addressMap.value[day.date] == true)
colors.secondaryVariant else Color.Transparent)
            )
        }
    }
}
```

By changing the features in the day class, we can customize our own design in terms of what we want to be labeled on the date as well as the date being highlighted.

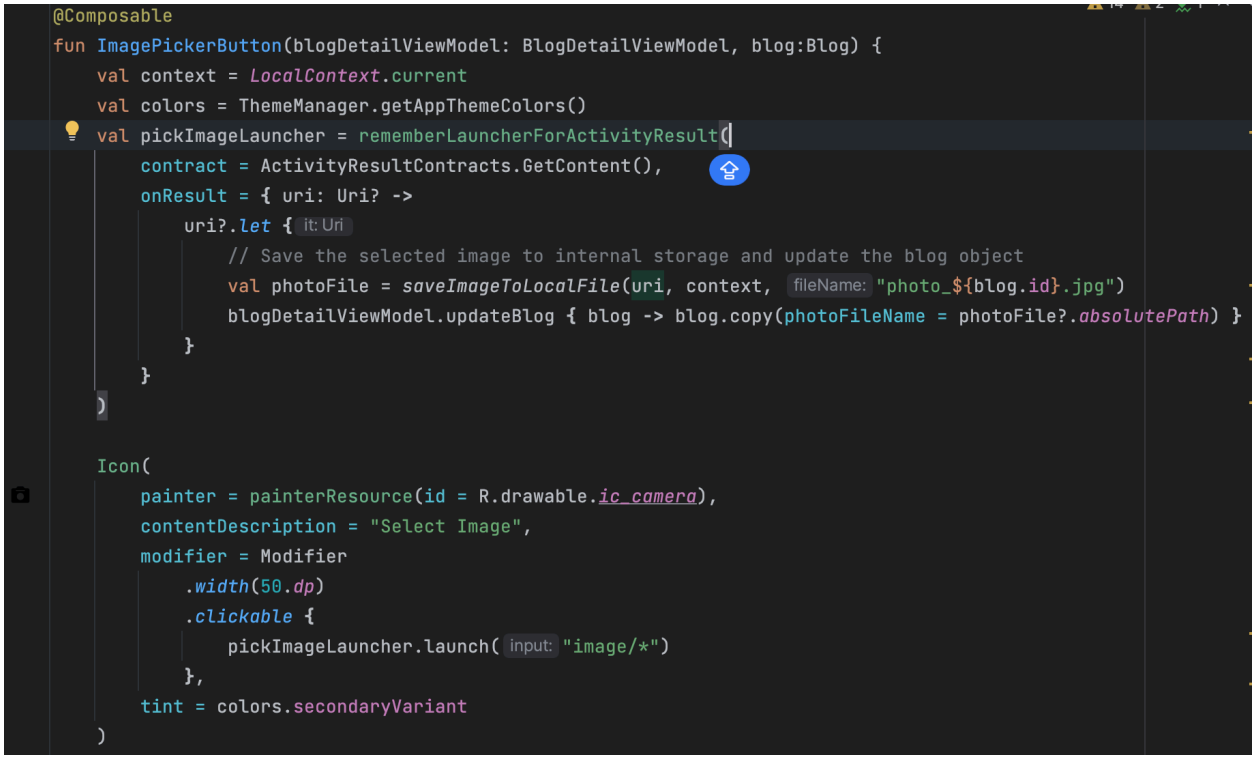
The way we did this is to build a map *MutableStateFlow<Map<LocalDate, Boolean>>* that would hold all the data which we have created and put it in a Viewmodel so that it will be updated when there are changes but also communicate through views.

The reason we need a map is to allow O(1) lookup time to ensure that the blog will not slow down even when we have a giant list of blogs.

In the Day class, we could then add the dot if a blog exist on that date. One thing we can improve is to change the color or size of the dot depending on the amount of blog written on that day. It will still be relatively essay when we have the map implemented.

Blog Detail Page:

1. All the UIs are completed using Jetpack Composable functions
2. A Navigator controller is used to go into a blog detail page from other pages, and also from detail page to map screen and others listed in the navigation bar.
3. A BlogDetailViewModel class and a factory class is created for UI to communicate with the database. For each different blog, a new BlogDetailViewModel instance would be created from the factory based on a given blog's UUID and navigate into it.
4. Emojis, locations, contents and uri of images are stored in the blogs and updated through BlogDetailViewModel. Emojis and contents are implemented with Textfields, others are icons with clickable activities.
5. Image files are stored locally. When the camera button is clicked, a launcher would appear to access the local album to choose local photos to upload. If you have already chosen an image, it would be replaced by the new one.



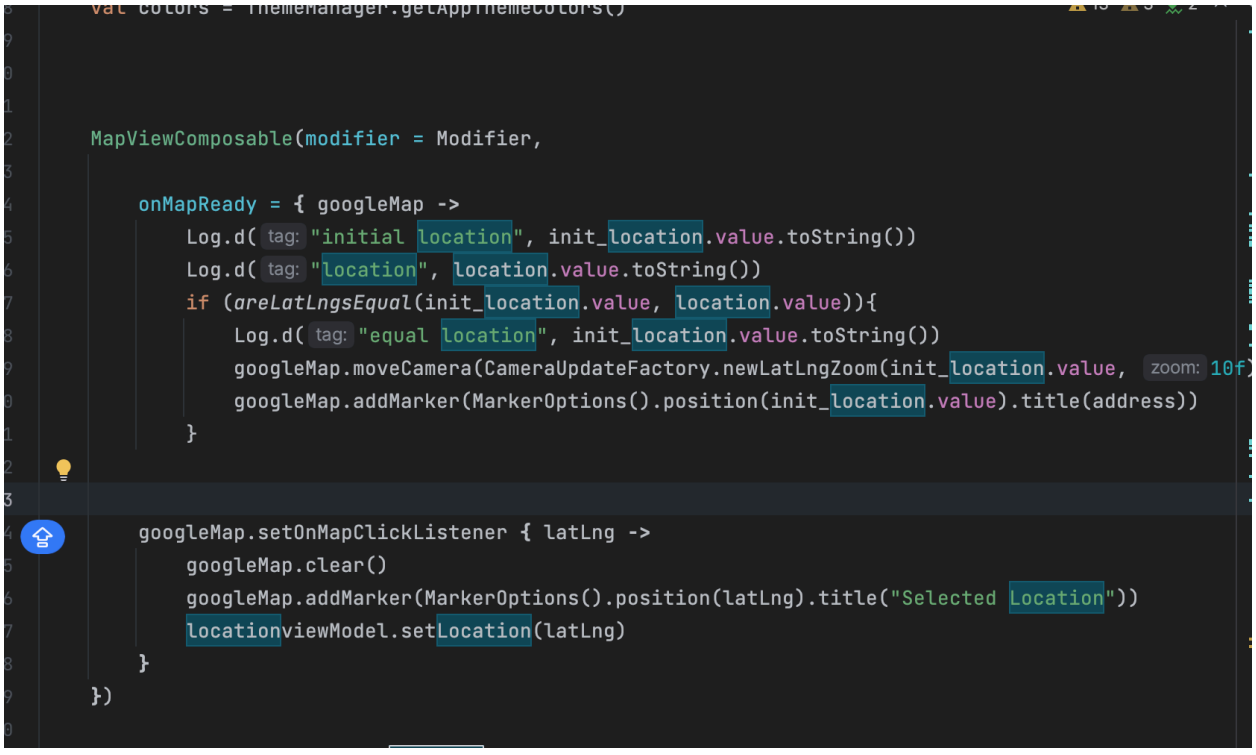
```
@Composable
fun ImagePickerButton(blogDetailViewModel: BlogDetailViewModel, blog:Blog) {
    val context = LocalContext.current
    val colors = ThemeManager.getAppThemeColors()
    val pickImageLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.GetContent(),
        onResult = { uri: Uri? ->
            uri?.let { it: Uri
                // Save the selected image to internal storage and update the blog object
                val photoFile = saveImageToLocalFile(uri, context, fileName: "photo_${blog.id}.jpg")
                blogDetailViewModel.updateBlog { blog -> blog.copy(photoFileName = photoFile?.absolutePath) }
            }
        }
    )

    Icon(
        painter = painterResource(id = R.drawable.ic_camera),
        contentDescription = "Select Image",
        modifier = Modifier
            .width(50.dp)
            .clickable {
                pickImageLauncher.launch( input: "image/*")
            },
        tint = colors.secondaryVariant
    )
```

Map:

1. A google map api is used to implement this. The API key is stored within local.properties and the name is called in manifest.
2. Google Mobile Services(gms).maps is widely used for composable functions to communicate with google map and translate information. Google map uses a LatLng type as a representation of latitude and longitude to record a location on the earth. A geocoder is used to convert a LatLng to a string of human friendly address format and

vice versa. The address is truncated to only the city, states and country to display in the list views or carousels.

3. A locationViewModel is used to get information from blogs and transit the information to map view composable functions to avoid the issues from coroutines. Every time when users click on the map, the locationViewModel would be updated and the map can get the information of the new location and reset the marker. Inside locationViewModel there is another variable called init_location which stores the initial location before user has set any location for a blog. It is set default the location of Boston University CDS building. So the marker and camera are placed here first. The BlogDetail page would tell the locationViewModel about the location stored in it to make sure the map would start with the stored location in the blog first. Otherwise it starts from BU CDS. During the map activity, since the location variable in LocationViewModel is updated each time the user clicks on the map, it would be different from init_location and at this moment, the camera won't be reset to the new location to prevent drastic change of the screen.
4. After the confirm button is clicked, the location would be stored in the blog through updating blogDetailViewModel and then navigate back to the last page to ensure the composable functions are also immediately updated.



Locations Screen:

1. Another screen for displaying all the locations in all the blogs are created in a similar way. In BlogListViewModel, the list of all the locations is stored and in Atlas page, all the locations are collected to avoid coroutine problems and are shown as markers on the

map. Then we utilized the gms.maps.model.LatLngBounds to help us get the best location and scale to put the camera to show all the markers on the map.

4. Future Work

The current version of our application already supports users in completing all designed operations locally. Moving forward, we can make enhancements and improvements in the following areas:

- Swipe between screens / swipe to delete
- Interactive Map
    - Select blogs by clicking on the markers on the map
    - Markers showing address information and pictures (like Apple photos)
- Multiple photos /locations in one blogs
- Bind with personal account (e.g. App account & Instagram)
- Cloud storage.
- UI polish.

5. Risks

One problem is that Jetpack Compose is constantly being updated and many function could be deprecated.

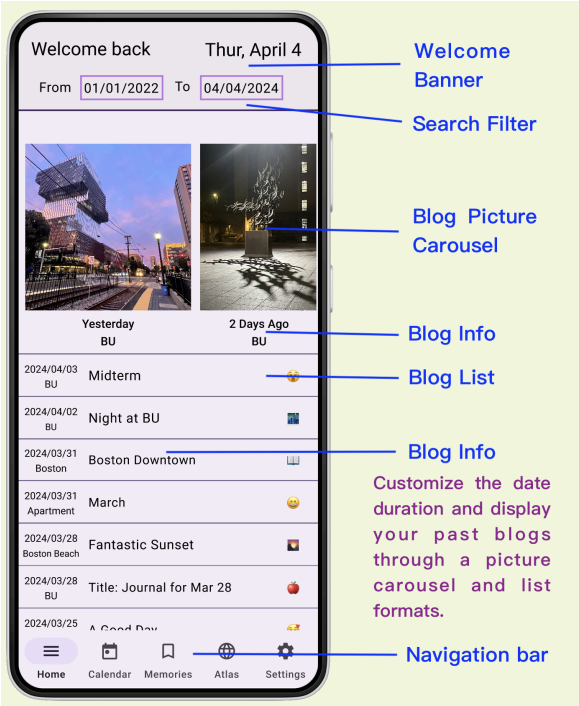
Another one would be the API configuration. If API key is not configured correctly, the APP will not work

6. Project Link

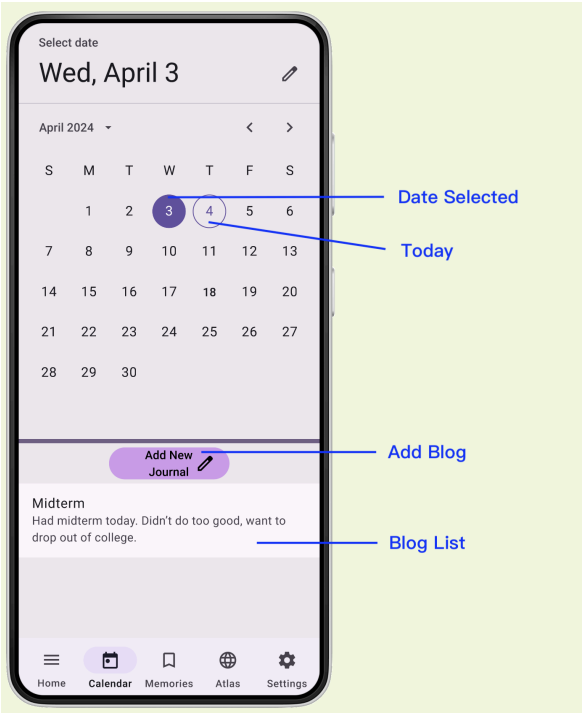
GitHub Link: https://github.com/HaoxuanSUN/WangLinSun_Project
Figma Design:
https://www.figma.com/file/Cecp129WQA0LLedprTBGN6/Material-3-Design-Kit-(Community)?type=design&node-id=11%3A1833&mode=design&t=ogvClAmbnnPwFpjv-1

7. Storyboard

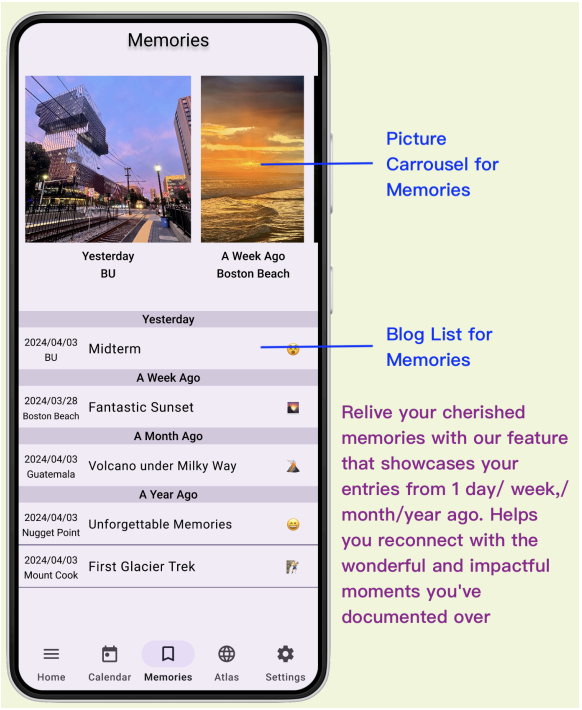
Following are the UI design and storyboard for the project (Using Figma).



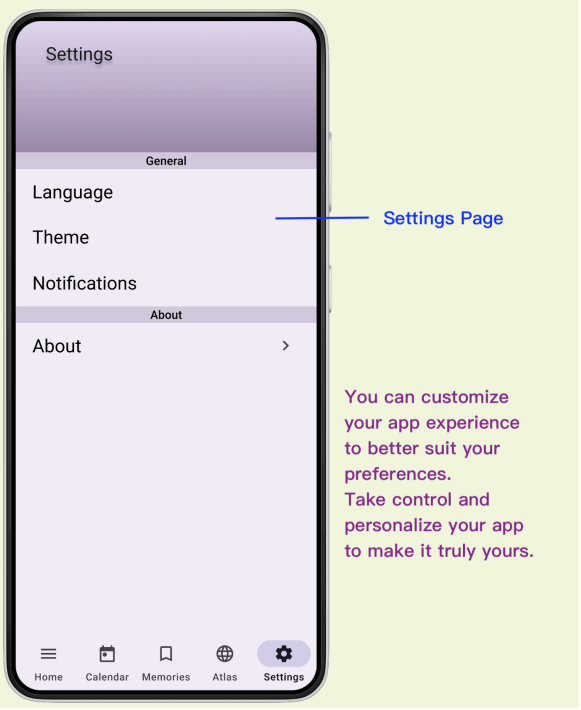
Home Page



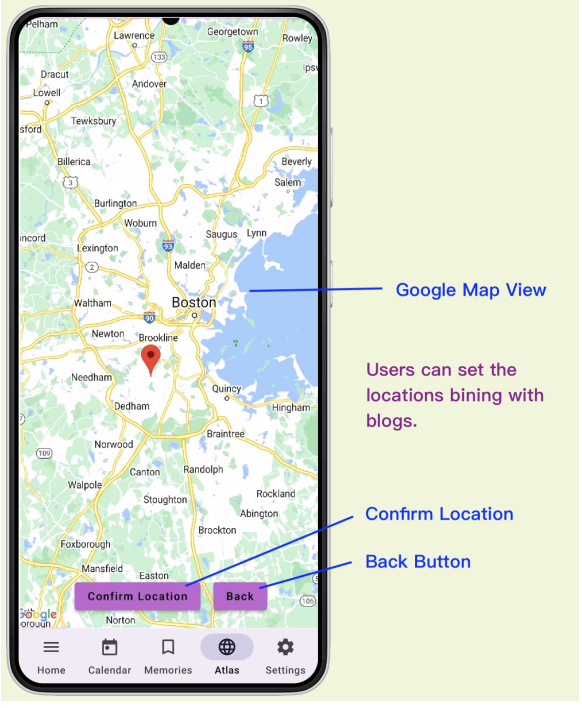
Calendar Page



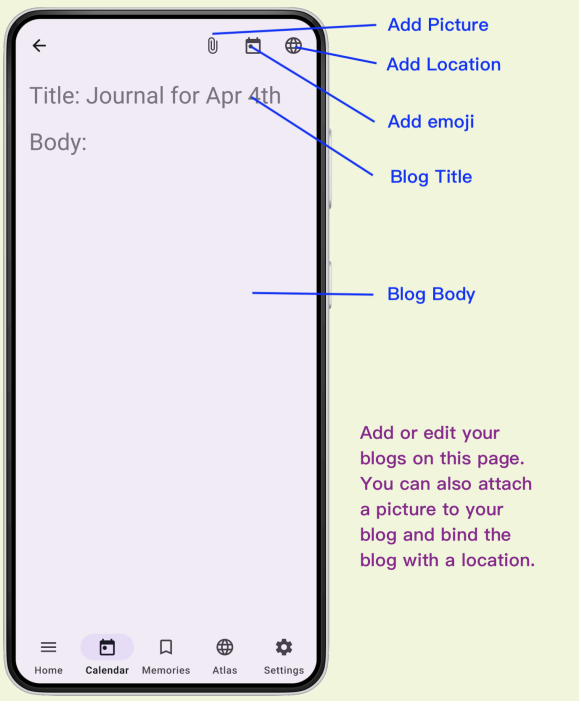
Memories Page



Settings Page



Locations Page



Blog Details Page

8. Bonus Points Achieved

Proper request of device permissions.

Proper interfaces for communication if using Fragments.

Use of RecyclerView or CardView where appropriate. (Using Compsoe LazyList and Card)

Good use of Menus. (Use NavigationBar)

Using Gestures and Accelerometer.

Targeting multiple locales (6 languages supported)

9. References

https://developers.google.com/maps/documentation/android-sdk/overview?hl=zh-cn
https://medium.com/@khater/swipeable-component-with-jetpack-compose-swipe-to-delete-or-archive-debfe4503613
https://developer.android.com/develop/ui/views/notifications/notification-permission
https://www.youtube.com/watch?v=iENK5EDO2iU&t=80s
https://www.youtube.com/watch?v=WouAQmqJI_I&t=316s
https://github.com/kizitonwose/Calendar
https://chatgpt.com/
https://developer.android.com